

Rubiks - Project Report

INF221

Zeno Elio Leonardi

Introduction

The Rubik's cube is a puzzle consisting of 6 faces, each made up of 9 coloured stickers, giving a total of 54 stickers. The physical parts creating a cube are categorised into three types: 8 corner cubies (3 stickers), 12 edge cubies (2stickers), and 6 center cubies (1 sticker). The cube has approximately 43 quintillion valid permutations, making it impossible for shitty laptops like mine to bruteforce solving a cube.

The cube is manipulated through 6 face moves: Right (R), Left (L), Up (U), Down (D), Front (F), and Back (B). Each move rotates the corresponding face 90° clockwise when that face is oriented to the right. In addition to the standard moves, each has a prime variant (e.g. R', L') representing a 90° counter-clockwise rotation, and a double variant (e.g. R2) representing a 180° rotation. Triple moves reduce to prime moves and are therefore not given separate notation, at least for the 3x3 cube.

The most popular method for solving Rubik's cubes is a method called CFOP. It stands for cross, F2L (first two layers), OLL (orientations of the last layer) and PLL (permutation of the last layer). Normally the cross is solved on the white side, making sure that all the edges line up with the centers. Then the first two layers are solved, either by first solving the face of the cube and then the middle edges, or doing both at the same time. OLL then solves the yellowface with the wrong positions of the cubies, which then PLL fixes. This solves the cube.

Description

Rubiks is a CLI for scrambling, exploring and solving the Rubik's cube in 2D. The project models a 3x3 Rubik's cube as a record of 8 corner cubies and a record 12 edge cubies.

For my assignment I wanted to create a virtual Rubik's cube inside of the terminal. I have been solving Rubik's cubes for a while now and wanted to give it a shot programmatically. The plan was initially to be able to insert a scrambled cube, and then make the program solve it for you iteratively. But inputting all the stickers manually every time wasn't the best user experience. Therefore I opted for combining the scrambler and solver in one! In addition the user can also try to solve it in the terminal themselves, but this is really hard since the visuals aren't that great.

At the time of writing this rapport the solver is not finished. I managed to dig myself into the never ending rabbit hole called heuristics. Both me and my computer have crashed on multiple occasions. Now on the last day I gave up and chose to solve the cube using the

same mental model as when I solved it myself. The code is ugly, but it works so I will finish this until the presentation. The rest of the project I'm really happy with. The CLI works great and it is a fun challenge to try to solve the cube using it (I have not managed to do so yet).

Once I'm done with the solver it would be really nice to use computer vision to be able to input a scrambled cube in a nice way. In addition it would be fun to solve some other cubes as well. I think the 2x2 would be a nice next step, but larger cubes would require a lot more work. Also I could improve on the graphics of the cube, but I was quite restricted on what I could do in the terminal.

Developing the project

This project fixes the cube's orientation so that the white face is always on top and the green face always faces the solver. This is a common default state used for scrambling the rubiks cube, and also helps with solving the cube programmatically.

When representing the cube as data there are many options. For example a list of stickers, as 6 faces, as 8 corners and 12 edges and as a matrix. I chose to go with the 8 corners and 12 edges. The center cubies are therefore fixed reference points and can be ignored when solving. Each cubie is modelled by a position and an orientation value. Corners have a 3-state orientation (0, 1, 2) and edges have a 2-state orientation (0, 1), capturing how much each piece has been twisted relative to its solved state. This was quite easy to model in Haskell using algebraic types.

In my program a Cube is represented as a record of corner cubies and a record of edge cubies. Every cubie has its home position (where it is in the solved state of the cube) and also an orientation as described above. The position of a corner cubie is represented at the coordinates where its solved position is. URF for example is the corner cubie which is in the up, right, front position. In the solved state this would be the white, green and red corner. This is the same for edges, but they only have two characters representing them i.e UF is the up, front cubie which is the white and green one.

After finishing the model of the cube, the next step was to implement the graphics of the cube. Since I wanted to use the terminal I used ANSI escape codes to color to space characters to a given color. This was a good second step because it let me see the cube and visualize the next feature which was the actual moves.

The implementation of moves went quite smoothly. This is because the cubie representation of the cube made it easy to keep all the moving stickers together. If I would have chosen for example the 6 face representation, I would have had to move all 18 stickers on each rotation, which would have been easier to mess up both the repositioning and orientation of the stickers. What the cubie representation allowed me to do was make a move spec type containing the rotations. The move spec contains two lists which say what corners and edges are involved in the rotation and also how their orientation changes. The edges orientation only changes for R and L moves while the corners orientations change for all

moves. Implementing the corners orientation was probably the most tricky functionality until this point.

After implementing all this I could finally implement the first real feature which is the scrambler. A official Rubik's cube scrambler uses the [TNoodle-WCA-1.2.3](#) scrambler, but this was a bit too complicated for me to replicate, so I chose to use a random moves with some rules:

1. Opposite moves can not happen after eachother i.e a R move can not happen after a L move.
2. Normal moves can not be followed by its prime, since this is the same as doing nothing.
3. The same move can not happen more than twice in a row, since 3 R is the same as one R'

Following these 3 rules and generating 25 random moves is how the scrambler works now.

By far the most difficult part was to actually solve the cube from a scrambled state. I think 90% of the time working on this project was spent on trying to solve it. I started with the naive solution, to just brute force moves to reach the different states needed to solve the cube with CFOP. This worked for the white cross, but not for anything else since it takes exponentially more time to reach each state. I then started to research on how to create lookup tables for the cube using heuristics. This proved to be the deepest rabbit hole I have ever been to. My laptop crashed on multiple occasions because of the solutions eating up all of my ram and I never got it working. So while writing this rapport the solver does only two things: solves the white cross and all the white corners. For me to do this I used the same solutions as when I solved it by hand, i.e. see where a cubie is and insert it into a slot using a hardcoded algorithm. Although this is really ugly it works, and I think this is what I should have done from the start.

Functional techniques

I think the most prominent functional technique I used was functional types to represent a cube. The entire cube model is built from algebraic data types. Move is a sum type with 18 variants (U | D | F | B | ...), making it impossible to represent an invalid move. CornerCubie is a product type pairing a CornerPosition with a CornerOrientation, and the cube itself is Cube a product Edges and corners. This means the type system enforces that the cube is correct and that Cube` always has exactly 8 corners and 12 edges. ADTs were a natural fit for this project since the physical structure of the puzzle fits directly onto sum and product types. I also used the IO monad to make the program interactive. This was necessary to make the CLI for the solver to read inputs and print outputs to the user.

Running the application

For this project I chose to go with Cabal. You can either run the application by using

`cabal run rubiks -- <command> [args]`

Which builds and runs it. Or running

`cabal build`

`cabal install`

Which builds the executable and then adds it as a command in your terminal.

The CLI supports four commands:

scramble [n]

This generates a random scramble of n moves (25 by default) and prints the sequence of moves and the resulting cube.

`rubiks scramble`

`rubiks scramble 10`

show

`rubiks show`

The most useless command, which just shows the rubiks cube in its solved state.

solve

Generates a 25-move scramble, prints the scrambled cube, then applies the solver and prints the cube after each solving phase of CFOP. For now it can only solve the white face of the cube. But I will implement the rest.

`rubiks show`

manual

Starts an interactive session where the user can attempt to solve a randomly scrambled cube themselves. The cube state is printed after each move. Enter moves using standard notation: R, L, U, D, F, B for clockwise face turns; append p for prime (e.g. Rp, Lp); append 2 for double turns (e.g. R2, U2). The session ends when the cube is solved.

`rubiks manual`

Evaluation

It was a really nice learning experience, I learned a lot about Haskell and really enjoyed working on this project since I have solved a lot of rubiks cubes. I would have focused more on the actual theory of functional programming and try to incorporate more of it instead of focusing so much on the actual solution of the Rubik's cube. There is a lot of repeating code

that I could have abstracted better. Also since I used so much time on the solver I did not have good time to write this report and it is therefore really badly written.